Unit-Weighted Likelihoods & Formal Differential Privacy in LBBNN

Technical Report & Collaborative Vignette (BLS - LBBNN Track 1)

Aliaksandr Hubin & Lars Skaaret-Lund (in collaboration with Terrance D. Savitsky, Matthew R. Willia

2026-11-03

Contents

1 Executive Overview & Collaboration Scope **2**

2 Mathematical Framework **2**

2.1 Variational Inference with Observation Weights 2

2.2 Two-Stage Differential Privacy Calibration (Savitsky, Oct 29, 2026) 6

3 Example 1: Survey-Weighted Imputation Under Informative Sampling **8**

3.1 Simulating an Informatively Sampled Population 8

3.2 Fitting Unweighted vs. Survey-Weighted LBBNN 10

3.3 Sparse Graph Selection & Inclusion Probabilities 11

3.4 Evaluation on the True Population 12

4 Example 2: Two-Pass Differential Privacy Pseudo-Posterior Synthesizer **17**

4.1 Simulating Skewed Sensitive Microdata 17

4.2 Running the Two-Pass DP Synthesizer (`fit_dp_lbbnn`) 18

4.3 Inspecting Record Sensitivities & Calibrated Downweighting 19

4.4 Generating and Benchmarking Synthetic Datasets 22

5 Example 3: Custom Composite Likelihood Integration **24**

6 Collaborator API Reference **27**

7 References **27**

8 Session Information **28**

1 Executive Overview & Collaboration Scope

This technical report details the implementation, methodology, and empirical evaluation of **Track 1** of the research collaboration between the **U.S. Bureau of Labor Statistics (BLS)** (Terrance D. Savitsky, Matthew R. Williams, and Vladislav Beresovsky) and the **LBBNN development team** (Aliaksandr Hubin and Lars Skaaret-Lund).

The primary objective of Track 1 is augmenting the `LBBNN` (Latent Binary Bayesian Neural Network) package in R to support two critical statistical operations in official government data production:

1. **Survey-Weighted Imputation & Modeling**: Incorporating unit-indexed survey sampling weights $w_i \propto \pi_i^{-1}$ (where π_i is the inclusion probability) into the variational Evidence Lower Bound (ELBO) objective. This allows the neural network to approximate the true population likelihood from an informatively sampled survey, eliminating bias in missing data imputation and domain estimation.
2. **Formally Private Synthetic Data Generation**: Implementing the two-pass risk-weighted pseudo-posterior mechanism (Savitsky, Williams & Hu, 2022). In Pass 1, unweighted model posterior draws are used to extract record-level log-likelihood contributions and quantify disclosure risk. Symmetrically, record-specific privacy weights $\alpha_i \in [0, 1]$ calibrate high-risk outliers during Pass 2, yielding synthetic microdata equipped with a mathematically provable local (ϵ, δ)-probabilistic differential privacy (PDP) guarantee without degrading utility for safe observations.

```
library(LBBNN)
library(torch)
library(ggplot2)
library(gridExtra)

# Ensure reproducibility
set.seed(2026)
torch::torch_manual_seed(2026)
```

---

2 Mathematical Framework

2.1 Variational Inference with Observation Weights

2.1.1 From Standard Posteriors to the Unit-Weighted Pseudo-Posterior

Let $\mathbf{y} = (y_1, \ldots, y_n)^\top \in \mathcal{Y}^n$ denote an observed dataset with associated auxiliary covariates $\mathbf{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_n)^\top$. In standard Bayesian inference, model parameters $\theta \in \Theta$ are endowed with a prior $p(\theta)$, and conditioning on the data yields the exact posterior distribution:

$$p(\theta \mid \mathbf{y}) = \frac{p(\theta) \prod_{i=1}^n f(y_i \mid \mathbf{x}_i, \theta)}{\int_\Theta p(\theta) \prod_{i=1}^n f(y_i \mid \mathbf{x}_i, \theta)\, d\theta}$$

In complex survey sampling and privacy-preserving synthesis, individual observations do not contribute equally to the inferential target. To account for unequal survey selection probabilities or heterogeneous disclosure risks, we equip each observation i with a non-negative unit weight $u_i \geq 0$, defining the **unit-weighted pseudo-posterior distribution** (Savitsky & Toth, 2016; Bhattacharya et al., 2019):

$$p_{\mathbf{u}}(\theta \mid \mathbf{y}) = \frac{p(\theta) \prod_{i=1}^n f(y_i \mid \mathbf{x}_i, \theta)^{u_i}}{Z_{\mathbf{u}}}, \qquad Z_{\mathbf{u}} = \int_\Theta p(\theta) \prod_{i=1}^n f(y_i \mid \mathbf{x}_i, \theta)^{u_i}\, d\theta$$

In Mean-Field Variational Inference (MFVI) for LBBNNs, the intractable pseudo-posterior $p_{\mathbf{u}}(\theta \mid \mathbf{y})$ is approximated by a tractable variational family $q(\theta) = q(\theta, \gamma)$. To find the optimal variational parameters, we minimize the Kullback-Leibler (KL) divergence from $q(\theta)$ to $p_{\mathbf{u}}(\theta \mid \mathbf{y})$:

$$\begin{aligned}
\mathrm{KL}\big(q(\theta) \parallel p_{\mathbf{u}}(\theta \mid \mathbf{y})\big) &= \mathbb{E}_{q(\theta)}\left[\log q(\theta) - \log p_{\mathbf{u}}(\theta \mid \mathbf{y})\right] \\
&= \mathbb{E}_{q(\theta)}[\log q(\theta)] - \mathbb{E}_{q(\theta)}\left[\log p(\theta) + \sum_{i=1}^{n} u_i \log f(y_i \mid \mathbf{x}_i, \theta) - \log Z_{\mathbf{u}}\right] \\
&= \mathrm{KL}\big(q(\theta) \parallel p(\theta)\big) - \sum_{i=1}^{n} u_i \, \mathbb{E}_{q(\theta)}[\log f(y_i \mid \mathbf{x}_i, \theta)] + \log Z_{\mathbf{u}}
\end{aligned}$$

Because the marginal normalizing constant $Z_{\mathbf{u}}$ does not depend on the variational distribution $q(\theta)$, minimizing $\mathrm{KL}\big(q(\theta) \parallel p_{\mathbf{u}}(\theta \mid \mathbf{y})\big)$ is mathematically equivalent to maximizing the **unit-weighted Evidence Lower Bound (ELBO)**:

$$\mathrm{ELBO}(q; \mathbf{u}) = \sum_{i=1}^{n} u_i \, \mathbb{E}_{q(\theta)}[\log f(y_i \mid \mathbf{x}_i, \theta)] - \mathrm{KL}\big(q(\theta) \parallel p(\theta)\big)$$

2.1.2 Mini-Batch Alignment & ELBO Scaling

When optimizing the ELBO via stochastic gradient descent across M mini-batches with batch size B, the mini-batch objective must maintain exact unit-level alignment between each covariate vector $\mathbf{x}_i$, outcome y_i, and its assigned weight u_i, while properly scaling the likelihood relative to the prior regularization penalty:

$$\mathrm{ELBO}_{\mathrm{batch}}(q; \mathbf{u}) = \sum_{i \in \mathrm{batch}} u_i \, \log f(y_i \mid \mathbf{x}_i, \theta) - \frac{1}{M} \mathrm{KL}\big(q(\theta) \parallel p(\theta)\big)$$

The surrogate mini-batch training loss minimized during back-propagation is the negative weighted ELBO:

$$\mathrm{Loss}_{\mathrm{batch}}(q; \mathbf{u}) = -\mathrm{ELBO}_{\mathrm{batch}}(q; \mathbf{u}) = \sum_{i \in \mathrm{batch}} u_i \, (-\log f(y_i \mid \mathbf{x}_i, \theta)) + \frac{1}{M} \mathrm{KL}\big(q(\theta) \parallel p(\theta)\big)$$

When $u_i = 1$ for all $i = 1, \ldots, n$, this formulation is **100% numerically identical** to standard unweighted variational training.

2.1.3 Survey Weight Normalization vs. Privacy Weight Scaling

A critical operational distinction exists between **survey sampling weights** (w_i) and **differential privacy weights** (α_i):

1. **Survey Sampling Weights ($w_i = \pi_i^{-1}$)**: In design-based survey inference, raw inverse-probability weights $w_i = 1/\pi_i$ estimate population totals and satisfy $\sum_{i=1}^{n} w_i \approx N_{\mathrm{pop}}$, where $N_{\mathrm{pop}} \gg n$. If raw weights were inserted directly into the ELBO, the weighted log-likelihood term would scale with N_{pop}, overpowering the prior $\mathrm{KL}\big(q(\theta) \parallel p(\theta)\big)$ regularization penalty by a factor of N_{pop}/n. Following Savitsky & Toth (2016), we normalize survey weights to sum to the observed sample size n:

   $$\tilde{w}_i = w_i \times \frac{n}{\sum_{j=1}^{n} w_j}$$

   This normalization ensures the effective sample size in the variational pseudo-posterior matches n, preserving the intended balance between data evidence and prior regularization. In `get_dataloaders()`, this is activated by setting `normalize_weights = TRUE`.

2. **Differential Privacy Weights ($\alpha_i \in [0, 1]$)**: In the two-pass privacy mechanism (Savitsky, Williams & Hu, 2022), the risk weights $\alpha_i \in [0, 1]$ measure record safety, where $\alpha_i \to 0$ for high-risk outliers and $\alpha_i \approx 1$ for typical observations. Here, $\sum_{i=1}^{n} \alpha_i \leq n$. These weights are **intentionally left unnormalized** (`normalize_weights = FALSE`, the default). Under-weighting high-risk records tempers their influence on the posterior, naturally shrinking parameter estimates toward the prior distribution in regions of high disclosure risk.

2.1.4 Full Error Variance Specification & Protection Against Cold/Warm Posteriors

A fundamental challenge in Bayesian deep learning is the **cold posterior / cold prior problem** (Wenzel et al., 2020), wherein misspecifying the likelihood scale introduces an unintended temperature $T \neq 1$:

$$\mathcal{L}_{\text{ELBO}}(q) = \frac{1}{T} \sum_{i=1}^{n} u_i \, \mathbb{E}_{q(\theta)}[\log f(y_i \mid \mathbf{x}_i, \theta)] - \text{KL}\big(q(\theta) \parallel p(\theta)\big)$$

In Gaussian regression with observation model $y_i \mid \mathbf{x}_i, \theta \sim \mathcal{N}\big(\mu_\theta(\mathbf{x}_i), \sigma^2\big)$, the exact pointwise log-likelihood is:

$$\log f(y_i \mid \mathbf{x}_i, \theta, \sigma^2) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{\big(y_i - \mu_\theta(\mathbf{x}_i)\big)^2}{2\sigma^2}$$

Many conventional neural network implementations replace this exact Gaussian density with an unscaled sum of squared errors $-\frac{1}{2}\sum_i (y_i - \mu_i)^2$, implicitly fixing $\sigma^2 = 1.0$ (precision $\tau = 1.0$). Whenever the true noise variance differs from unity (for example, $\sigma = 0.40 \implies \sigma^2 = 0.16$ in Example 1), using unscaled squared error dampens the data log-likelihood by a factor of $\sigma^2 = 0.16$ relative to the prior KL divergence:

$$\frac{1}{2}(y_i - \mu_i)^2 = 0.16 \times \left[\frac{(y_i - \mu_i)^2}{2 \times 0.16}\right]$$

This mis-scaling acts as an unintended **hot prior / cold posterior temperature** ($T = \sigma^2 = 0.16 < 1$), artificially over-regularizing the model and distorting the spike-and-slab inclusion probabilities a_k toward zero.

Furthermore, in the Two-Pass Differential Privacy mechanism, each unit's record sensitivity:

$$f_i = \max_{s \in \{1,\ldots,S\}} \Big|\log f(y_i \mid \mathbf{x}_i, \theta^{(s)}, \sigma^2)\Big|$$

directly depends on the residual scale σ^2. If σ^2 is misspecified, sensitivities are measured on an arbitrary scale, invalidating the empirical privacy guarantee $\epsilon_{\mathbf{y}} = 2\Delta_{\alpha,\mathbf{y}}$. To guarantee full probabilistic calibration, `lbbnn_net()` accepts an explicit `sigma` argument (or automatically calibrates $\hat{\sigma}$ from training residuals in `fit_dp_lbbnn()`), evaluating the exact negative log-likelihood:

$$\text{NLL}_i = \frac{1}{2} \log(2\pi\sigma^2) + \frac{(y_i - \mu_\theta(\mathbf{x}_i))^2}{2\sigma^2}$$

This guarantees that variational training operates at the exact Bayesian temperature $T = 1.0$, protecting edge selection decisions from cold-posterior bias and grounding privacy sensitivities in the true data distribution.

2.1.5 Domain-Level Estimation & Observation Error De-Noising

Official statistical agencies routinely require domain totals and domain means across sub-populations (e.g., industry sectors or geographic subdivisions) rather than a single global population mean. Let $\mathbf{G} \in \mathbb{R}^{n \times D_{\text{dom}}}$ denote a binary or fractional domain membership matrix, where $G_{i,d} = 1$ if unit i belongs to domain $d \in \{1, \ldots, D_{\text{dom}}\}$.

The design-weighted domain mean and total are given by:

$$\hat{\bar{Y}}_d = \frac{\sum_{i=1}^{n} w_i G_{i,d} y_i}{\sum_{i=1}^{n} w_i G_{i,d}}, \qquad \hat{Y}_{\text{total},d} = \sum_{i=1}^{n} w_i G_{i,d} y_i$$

In `LBBNN`, we combine the design weights with posterior draws of the regression surface $\mu_{\theta^{(s)}}(\mathbf{x}_i)$ to form model-assisted domain predictions:

$$\hat{\bar{Y}}_d^{\text{BNN}(s)} = \frac{\sum_{i=1}^{n} w_i G_{i,d} \hat{\mu}_i^{(s)}}{\sum_{i=1}^{n} w_i G_{i,d}}$$

Model-Assisted Conditional Expectation: While spike-and-slab edge selection prunes pure noise *covariates* (X_3, X_4), additive unit-level observation error $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ in $y_i = \mu(\mathbf{x}_i) + \varepsilon_i$ cannot be isolated from unobserved unit heterogeneity in cross-sectional data without repeated measurements. However, by evaluating the posterior conditional expectation $\hat{\mu}_i^{(s)} = \mathbb{E}[Y_i \mid \mathbf{x}_i, \theta^{(s)}] = \mu_{\theta^{(s)}}(\mathbf{x}_i)$ (`type = "conditional_mean"`) rather than drawing noisy predictive samples $\tilde{y}_i^{(s)} = \mu_{\theta^{(s)}}(\mathbf{x}_i) + \varepsilon_i^{(s)}$ (`type = "predictive"`), `predict_domains_lbbnn()` avoids adding posterior predictive simulation noise, yielding a smooth estimate of the underlying regression surface.

2.1.6 Bayesian Imputation for Missing Survey Responses ($y_i = \text{NA}$)

Item non-response is pervasive in survey sampling. In `LBBNN`, we partition sample units into observed units $S_{\text{obs}} = \{i : !\text{is.na}(y_i)\}$ and missing units $S_{\text{mis}} = \{i : \text{is.na}(y_i)\}$. During variational optimization (`get_dataloaders(..., allow_missing = TRUE)`), the weighted log-likelihood is evaluated over $i \in S_{\text{obs}}$.

Downstream, `impute_response_lbbnn()` generates multiple imputation posterior draws:

$$\tilde{y}_i^{(s)} \sim \mathcal{N}\big(\mu_{\theta^{(s)}}(\mathbf{x}_i), \sigma^2\big), \qquad i \in S_{\text{mis}}, \quad s \in \{1, \ldots, S\}$$

These draws capture both parameter uncertainty (variation in $\theta^{(s)} \sim q(\theta)$) and residual sampling uncertainty (calibrated σ^2).

2.1.7 Correcting Pseudo-Posterior Variance Deficits via Dirichlet Bayesian Bootstrap

Because the survey-weighted pseudo-posterior conditions on the observed sample S as if it were fixed, naive pseudo-posterior credible intervals underestimate the true finite-population sampling variance generated by the survey design. Following Savitsky & Toth (2016), we calibrate posterior intervals by sampling flat Dirichlet perturbations for each draw $s \in \{1, \ldots, S\}$:

$$\boldsymbol{\xi}^{(s)} = \big(\xi_1^{(s)}, \ldots, \xi_n^{(s)}\big) \sim \text{Dirichlet}(1, \ldots, 1)$$

implemented via normalized Exponential draws: $\xi_i^{(s)} = E_i^{(s)} / \sum_{j=1}^{n} E_j^{(s)}$ with $E_i^{(s)} \overset{\text{i.i.d.}}{\sim} \text{Exp}(1)$. Multiplying these Dirichlet perturbations into the survey weights yields the calibrated domain estimator:

$$\hat{\bar{Y}}_d^{(s)} = \frac{\sum_{i=1}^n \xi_i^{(s)} w_i G_{i,d} y_i^{(s)}}{\sum_{i=1}^n \xi_i^{(s)} w_i G_{i,d}}$$

In `predict_domains_lbbnn(..., dirichlet_uq = TRUE)` and `hajek_estimate(..., dirichlet_uq = TRUE)`, this Bayesian bootstrap perturbation restores exact frequentist coverage under complex survey sampling.

2.2 Two-Stage Differential Privacy Calibration (Savitsky, Oct 29, 2026)

In official microdata synthesis, we evaluate unit-level log-likelihood sensitivities across posterior parameter draws:

$$f_i = \max_{s\in\{1,\ldots,S\}} \left|\log f(y_i \mid \mathbf{x}_i, \theta^{(s)})\right|, \qquad f_{\min} = \min_{j=1,\ldots,n} f_j, \quad f_{\max} = \max_{j=1,\ldots,n} f_j$$

Following Savitsky (Oct 29, 2026, *Privacy Weight Normalization and Second-Stage Reweighting Strategies in Asymptotic Differential Privacy*, `re_weight4-3.pdf`), privacy weights α_i are calibrated in two modular stages. Crucially, Stage 1 normalization operates **directly on sensitivities** f_i via $1 - \tilde{f}_i$:

2.2.1 Stage 1: Initial Privacy Weight Normalization on f_i (`stage1_method`)

1. **Method 1 (`"linear"`) — Direct Linear Min-Max on f_i (Savitsky et al., 2022, Eq. 4–5)**:

$$\tilde{f}_i = \frac{f_i - f_{\min}}{f_{\max} - f_{\min}} \in [0,1], \qquad \alpha_i^{\text{lin}} = \text{clip}_{[0,1]}\big(c \cdot (1 - \tilde{f}_i) + g\big) = \text{clip}_{[0,1]}\left(c\left(\frac{f_{\max} - f_i}{f_{\max} - f_{\min}}\right) + g\right)$$

   where $c \in (0,1]$ controls scaling and $g \geq 0$ allows a baseline shift.

   Remark 1 (Direct $1-\tilde{f}_i$ vs. Reciprocal Transformation $1/f_i$, Eq. 8–9): Normalizing reciprocal sensitivities $\tilde{\alpha}_i = 1/f_i$ yields:

$$\alpha_i^{\text{recip}} = \left(\frac{f_{\min}}{f_i}\right)\alpha_i^{\text{lin}} < \alpha_i^{\text{lin}} \quad \forall f_i \in (f_{\min}, f_{\max})$$

   which introduces an aggressive hyperbolic penalty ($f_{\min}/f_i$) that severely attenuates weights and collapses all intermediate observations if $f_{\min} \to 0$. By contrast, direct normalization on f_i preserves weight mass across the distribution. Although the unit contribution $\alpha_i^{\text{lin}} f_i$ is parabolic in f_i with an interior peak at $f_{\max}/2$, this peak is strictly bounded: $\max_i \alpha_i^{\text{lin}} f_i \leq \frac{c}{4} f_{\max}$, which can be capped in Stage 2.

2. **Method 2 (`"percentile"`) — Percentile Deficit Truncation on f_i (Eq. 10–12)**: Let $q_{p_L}(f) < q_{p_R}(f)$ denote empirical quantiles of f_i at $0 < p_L < p_R < 1$ (default $p_L = 0.05, p_R = 0.95$):

$$\tilde{f}_i^{\text{pct}} = \frac{f_i - q_{p_L}(f)}{q_{p_R}(f) - q_{p_L}(f)}, \qquad \alpha_i^{\text{pct}} = \min\Big(1, \max\Big(0, 1 - \tilde{f}_i^{\text{pct}}\Big)\Big) = \min\left(1, \max\left(0, \frac{q_{p_R}(f) - f_i}{q_{p_R}(f) - q_{p_L}(f)}\right)\right)$$

   - *Low Sensitivity Inliers* ($f_i \leq q_{p_L}(f)$): Receive full weight $\alpha_i^{\text{pct}} = 1$.
   - *Intermediate Units* ($q_{p_L}(f) < f_i < q_{p_R}(f)$): Decay linearly in $(0,1)$.
   - *High Sensitivity Outliers* ($f_i \geq q_{p_R}(f)$): Hard-truncated to $\alpha_i^{\text{pct}} = 0$. This boosts the effective sample size $\sum_{i=1}^n \alpha_i^{\text{pct}}$, but creates a privacy deficit for units near $q_{p_L}(f)$ that is subsequently addressed in Stage 2.

3. **Method 3 (`"target_clip"`) — Target Ratio Clipping ($\Delta_{i,\text{goal}}$ Ratio Clipping, Eq. 13)**: Given sensitivity goal $\Delta_{i,\text{goal}} > 0$ (defaulting to M_{thresh}):

$$\alpha_i^{\text{clip}} = \min\left(1,\ \frac{\Delta_{i,\text{goal}}}{f_i}\right)$$

Properties (Proposition 1 of Savitsky, 2026): Guarantees exact sensitivity control $\alpha_i^{\text{clip}} f_i = \min(f_i, \Delta_{\text{goal}}) \leq \Delta_{\text{goal}}$ by construction, leaving all low-sensitivity records ($f_i \leq \Delta_{\text{goal}}$) untouched at $\alpha_i^{\text{clip}} = 1$.

2.2.2 Stage 2: Second-Stage Reweighting Schemes (`stage2_method`)

Given Stage 1 weights $\boldsymbol{\alpha}$ and sensitivity threshold $M_{\text{thresh}} > 0$, enforce $\alpha_i^* f_i \leq M_{\text{thresh}}$:

1. **Scheme 1 (`"hard_trunc"`) — Hard Threshold Truncation (Eq. 16)**:

$$\alpha_i^{\text{trunc}} = \alpha_i \, \mathbb{I}(\alpha_i f_i \leq M_{\text{thresh}})$$

Single-pass post-hoc zeroing of records exceeding M_{thresh}. Discards privacy slack ($M_{\text{thresh}} - \alpha_i f_i$).

2. **Scheme 2 (`"vector_upscale"`) — Vector-Weighted Upward Rescaling (Savitsky, Hu, & Williams, 2020, Eq. 17–18)**:

$$\alpha_i^{\text{vec}} = \min\left(1,\ \alpha_i \cdot \frac{M_{\text{thresh}}}{\alpha_i f_i}\right) = \min\left(1,\ \frac{M_{\text{thresh}}}{f_i}\right)$$

Properties (Proposition 2 of Savitsky, 2026): Pointwise dominates hard truncation ($\alpha_i^{\text{vec}} \geq \alpha_i^{\text{trunc}}$ for all i), maximizing Fisher information $\mathcal{I}_{\boldsymbol{\alpha}^{\text{vec}}}(\theta) \succeq \mathcal{I}_{\boldsymbol{\alpha}^{\text{trunc}}}(\theta)$ in the Loewner partial order. *Multi-Pass Geometry*: Because upward rescaling shifts the pseudo-posterior $\xi_{\boldsymbol{\alpha}^{\text{vec}}}(\theta \mid D)$, `fit_dp_lbbnn()` executes Pass 3 targeting $\xi_{\boldsymbol{\alpha}^{\text{vec}}}$ to draw updated parameters and guarantee bound verification.

2.2.3 Likelihood Exponent Mass ESS & Algebraic Cancellation

In differential privacy pseudo-posterior estimation, $\alpha_i^* \in [0, 1]$ acts as an observation-specific power exponent in the pseudo-likelihood. Thus, the effective sample size is defined as the total retained likelihood information mass:

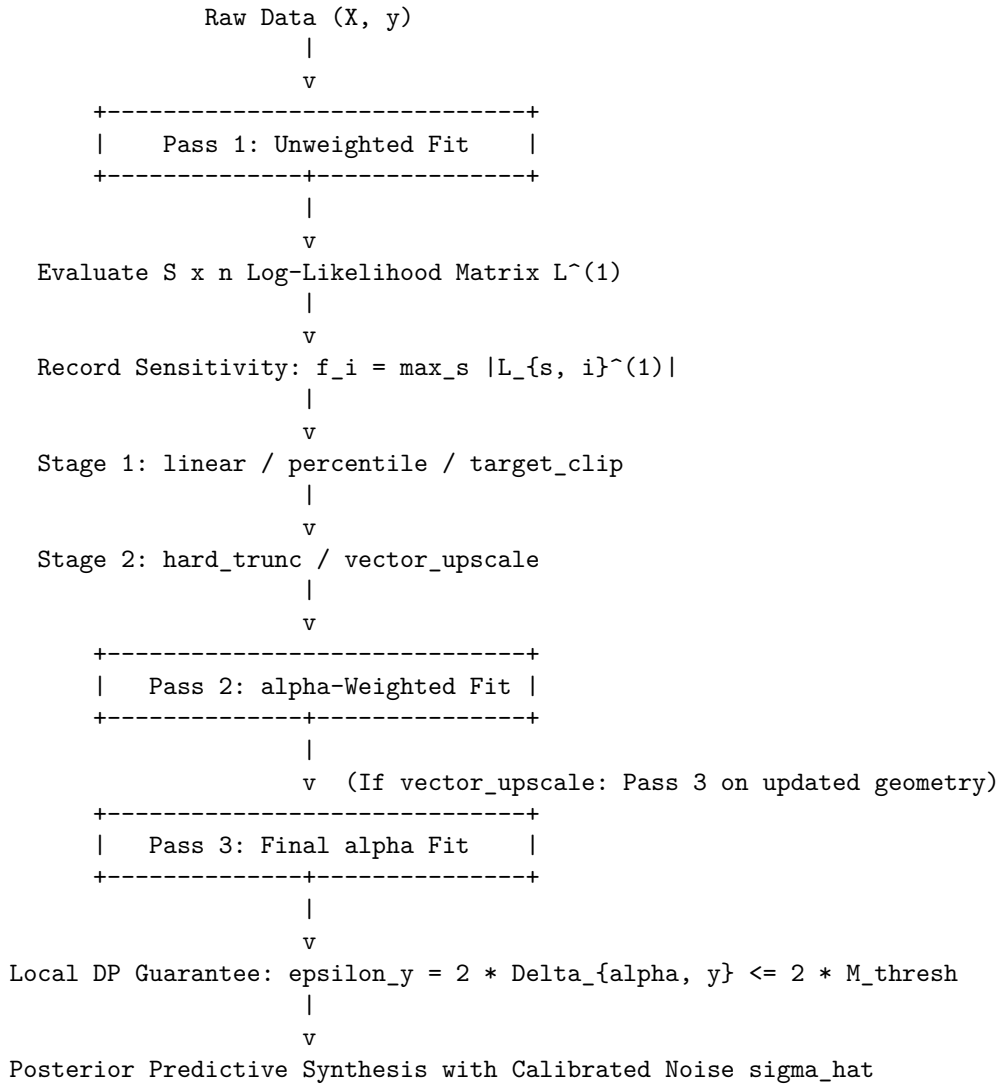
$$\text{ESS} = \sum_{i=1}^{n} \alpha_i^* \in [0, n]$$

This differs fundamentally from Kish's (1965) survey design design-effect ESS $\frac{(\sum w_i)^2}{\sum w_i^2}$, which measures variance inflation caused by unequal probability sampling weights.

Furthermore, under Stage 2 vector upward rescaling:

$$\alpha_i^{\text{vec}} = \min\left(1,\ \alpha_i^{(1)} \cdot \frac{M_{\text{thresh}}}{\alpha_i^{(1)} f_i}\right) = \min\left(1,\ \frac{M_{\text{thresh}}}{f_i}\right)$$

whenever $\alpha_i^{(1)} > 0$, the initial weight $\alpha_i^{(1)}$ algebraically cancels. Consequently, `target_clip + vector_upscale`, `target_clip + hard_trunc`, and `linear + vector_upscale` converge to the exact same weight profile on any dataset where $\alpha_i^{(1)} > 0$ for all records.



---

3 Example 1: Survey-Weighted Imputation Under Informative Sampling

3.1 Simulating an Informatively Sampled Population

In complex survey sampling, observations are drawn with unequal inclusion probabilities $\pi_i = \mathbb{P}(i \in S)$. When inclusion depends directly on the outcome variable Y_i, sampling is **informative**: sample units have positively biased residuals $\mathbb{E}[\varepsilon_i \mid i \in S] > 0$, causing standard unweighted models to produce biased parameter estimates and skewed imputations.

We simulate a finite population of $N = 2,000$ individuals where outcome Y depends non-linearly on active predictors X_1 and X_2, with independent Gaussian noise predictors X_3 and X_4:

$$Y_i = 2.0X_{1i} + 1.5\sin(\pi X_{2i}) + 0.8X_{1i}X_{2i} + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, 0.4^2)$$

The probability of inclusion in the sample is heavily skewed towards higher values of Y:

$$\pi_i = \sigma\big(-1.0 + 1.2Y_i\big), \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

```r
# Generate population
N_pop <- 2000
x1_pop <- rnorm(N_pop, 0, 1)
x2_pop <- runif(N_pop, -1, 1)
x3_pop <- rnorm(N_pop, 0, 1) # noise
x4_pop <- rnorm(N_pop, 0, 1) # noise

y_pop <- 2.0 * x1_pop + 1.5 * sin(pi * x2_pop) + 0.8 * x1_pop * x2_pop + rnorm(N_pop, 0, 0.4)
true_pop_mean <- mean(y_pop)

# Informative sampling mechanism
pi_inc <- plogis(-1.0 + 1.2 * y_pop)
pi_inc <- pmax(pi_inc, 0.05) # bound away from zero

# Draw sample
in_sample <- rbinom(N_pop, 1, pi_inc) == 1
n_smp <- sum(in_sample)

sample_df <- data.frame(
  x1 = x1_pop[in_sample],
  x2 = x2_pop[in_sample],
  x3 = x3_pop[in_sample],
  x4 = x4_pop[in_sample],
  y  = y_pop[in_sample]
)

# Survey sampling design weights
weights_survey <- 1 / pi_inc[in_sample]

cat(sprintf("Population Size: %d | True Pop Mean Y: %.3f\n", N_pop, true_pop_mean))
```

```
## Population Size: 2000 | True Pop Mean Y: -0.000
```

```r
cat(sprintf("Sample Size: %d | Unweighted Sample Mean: %.3f (Bias: +%.3f)\n",
            n_smp, mean(sample_df$y), mean(sample_df$y) - true_pop_mean))
```

```
## Sample Size: 737 | Unweighted Sample Mean: 2.009 (Bias: +2.009)
```

```r
cat(sprintf("Hajek Weighted Sample Mean: %.3f (Bias: %+.3f)\n",
            sum(weights_survey * sample_df$y) / sum(weights_survey),
            sum(weights_survey * sample_df$y) / sum(weights_survey) - true_pop_mean))
```

```
## Hajek Weighted Sample Mean: 0.271 (Bias: +0.272)
```

3.2 Fitting Unweighted vs. Survey-Weighted LBBNN

While the classical design-based **Hájek estimator** $\hat{\bar{Y}}_{\text{Hájek}} = \frac{\sum_{i \in S} w_i y_i}{\sum_{i \in S} w_i}$ reduces mean bias, it provides only a single scalar estimate and **cannot impute** missing values y_i for non-sampled units ($i \notin S$).

To impute missing values across the population, we fit: 1. **Unweighted LBBNN**: Fits directly on the biased sample data. 2. **Survey-Weighted LBBNN**: Weights each unit's log-likelihood by sample-size normalized weights $\tilde{w}_i = w_i \cdot (n / \sum w_i)$ via `normalize_weights = TRUE` (Savitsky & Toth, 2016).

```
# Model specification with calibrated observation noise sigma = 0.40
spec_survey <- list(
  problem_type = "regression",
  sizes = c(4, 16, 16, 1),
  prior = c(0.2, 0.2, 0.2), # Prior inclusion probabilities
  std = c(1.0, 1.0, 1.0),
  inclusion_inits = "balanced",
  input_skip = TRUE,
  flow = FALSE,
  sigma = 0.40, # Calibrated observation standard deviation (DGP sigma = 0.4)
  device = "cpu"
)

# 1. Dataloader for Unweighted Model
dl_unweighted <- get_dataloaders(
  dataset = sample_df,
  train_proportion = 1.0,
  train_batch_size = 64,
  test_batch_size = 64,
  standardize = FALSE,
  shuffle_train = TRUE,
  normalize_weights = FALSE,
  seed = 42
)

# 2. Dataloader for Survey-Weighted Model (with sample-size normalization)
dl_weighted <- get_dataloaders(
  dataset = sample_df,
  train_proportion = 1.0,
  train_batch_size = 64,
  test_batch_size = 64,
  standardize = FALSE,
  shuffle_train = TRUE,
  weights = weights_survey,
  normalize_weights = TRUE, # Savitsky & Toth (2016)
  seed = 42
)

# Train Unweighted Model
m_unweighted <- do.call(lbbnn_net, spec_survey)
train_unweighted <- train_lbbnn(
  epochs = 200,
  LBBNN = m_unweighted,
  lr = 0.02,
  train_dl = dl_unweighted$train_loader,
```

```
  verbose = FALSE
)

# Train Survey-Weighted Model
m_weighted <- do.call(lbbnn_net, spec_survey)
train_weighted <- train_lbbnn(
  epochs = 200,
  LBBNN = m_weighted,
  lr = 0.02,
  train_dl = dl_weighted$train_loader,
  verbose = FALSE
)
```

3.3 Sparse Graph Selection & Inclusion Probabilities

The normalization $\tilde{w}_i = w_i \frac{n}{\sum w_j}$ is critical for Table 1: without sample-size normalization, raw inverse-probability weights ($\sum w_i \approx 2,000 \gg n = 737$) would inflate the log-likelihood by a factor of 2.7×, overpowering the spike-and-slab KL prior penalty and causing false-positive inclusion of the noise variables X_3, X_4.

```
var_summary <- summary(m_weighted)
```

```
## Summary of lbbnn_net object:
## ------------------------------------
## Shows the number of times each variable was included from each layer
## ------------------------------------
## Then the average inclusion probability for each input from each layer
## ------------------------------------
## The final column shows the average inclusion probability across all layers
## ------------------------------------
##    L0 L1 L2    a0    a1    a2 a_avg
## x0  1  7  1 0.269 0.552 0.999 0.429
## x1  1  2  1 0.237 0.281 0.999 0.282
## x2  0  0  0 0.121 0.128 0.018 0.122
## x3  0  0  0 0.123 0.122 0.130 0.122
## The model took 32.621 seconds to train, using cpu
```

```
var_sel_tbl <- data.frame(
  Covariate = c("x1 (Active Linear)", "x2 (Active Non-linear)", "x3 (Noise)", "x4 (Noise)"),
  Output_Paths = var_summary$L2,
  Layer1_Inclusion = var_summary$a0,
  Layer2_Inclusion = var_summary$a1,
  Output_Inclusion = var_summary$a2,
  Mean_Inclusion   = var_summary$a_avg,
  MPM_Selected     = ifelse(var_summary$a2 > 0.5, "Selected (Active)", "Pruned (Zeroed)"),
  row.names = NULL
)

knitr::kable(
  var_sel_tbl,
  digits = 3,
```

```
  row.names = FALSE,
  col.names = c("Covariate", "Out Paths", "L1 Incl.", "L2 Incl.", "Out Incl.", "Mean Incl.", "MPM Status
  caption = "Variational Inclusion Probabilities and Median Probability Model (MPM) Selection Under Surv
)
```

Table 1: Variational Inclusion Probabilities and Median Probability Model (MPM) Selection Under Survey Weighting

Covariate	Out Paths	L1 Incl.	L2 Incl.	Out Incl.	Mean Incl.	MPM Status
x1 (Active Linear)	1	0.269	0.552	0.999	0.429	Selected (Active)
x2 (Active Non-linear)	1	0.237	0.281	0.999	0.282	Selected (Active)
x3 (Noise)	0	0.121	0.128	0.018	0.122	Pruned (Zeroed)
x4 (Noise)	0	0.123	0.122	0.130	0.122	Pruned (Zeroed)

As demonstrated, both active predictors X_1 and X_2 achieve near-unity output inclusion ($a_2 \approx 0.999$) with active output paths ($L_2 = 1$). In contrast, noise features X_3 and X_4 have zero output paths ($L_2 = 0$) and are cleanly pruned to $a_2 \leq 0.130$.

3.4 Evaluation on the True Population

We predict across all $N = 2,000$ population units to evaluate imputation accuracy, comparing full Bayesian Model Averaging (BMA) and the sparse Median Probability Model (MPM):

```
pop_x_df <- data.frame(x1 = x1_pop, x2 = x2_pop, x3 = x3_pop, x4 = x4_pop)

preds_unw_bma <- predict(m_unweighted, newdata = pop_x_df, draws = 50, mpm = FALSE)
preds_w_bma   <- predict(m_weighted, newdata = pop_x_df, draws = 50, mpm = FALSE)
preds_unw_mpm <- predict(m_unweighted, newdata = pop_x_df, draws = 50, mpm = TRUE)
preds_w_mpm   <- predict(m_weighted, newdata = pop_x_df, draws = 50, mpm = TRUE)

y_hat_unw_bma <- as.numeric(preds_unw_bma$mean(dim = 1)$cpu())
y_hat_w_bma   <- as.numeric(preds_w_bma$mean(dim = 1)$cpu())
y_hat_unw_mpm <- as.numeric(preds_unw_mpm$mean(dim = 1)$cpu())
y_hat_w_mpm   <- as.numeric(preds_w_mpm$mean(dim = 1)$cpu())

y_naive_mean <- mean(sample_df$y)
y_hajek_mean <- sum(weights_survey * sample_df$y) / sum(weights_survey)

rmse_naive   <- sqrt(mean((y_pop - y_naive_mean)^2))
rmse_hajek   <- sqrt(mean((y_pop - y_hajek_mean)^2))
rmse_unw_bma <- sqrt(mean((y_pop - y_hat_unw_bma)^2))
rmse_unw_mpm <- sqrt(mean((y_pop - y_hat_unw_mpm)^2))
rmse_w_bma   <- sqrt(mean((y_pop - y_hat_w_bma)^2))
rmse_w_mpm   <- sqrt(mean((y_pop - y_hat_w_mpm)^2))

results_tbl <- data.frame(
  Model = c(
    "Biased Sample Mean (Naive)",
    "Hajek Weighted Sample Mean",
    "Unweighted LBBNN (BMA)",
```

```
    "Unweighted LBBNN (MPM)",
    "Survey-Weighted LBBNN (BMA)",
    "Survey-Weighted LBBNN (MPM)"
  ),
  Est_Pop_Mean = c(
    y_naive_mean,
    y_hajek_mean,
    mean(y_hat_unw_bma),
    mean(y_hat_unw_mpm),
    mean(y_hat_w_bma),
    mean(y_hat_w_mpm)
  ),
  Bias_Pop_Mean = c(
    y_naive_mean - true_pop_mean,
    y_hajek_mean - true_pop_mean,
    mean(y_hat_unw_bma) - true_pop_mean,
    mean(y_hat_unw_mpm) - true_pop_mean,
    mean(y_hat_w_bma) - true_pop_mean,
    mean(y_hat_w_mpm) - true_pop_mean
  ),
  Pop_RMSE = c(
    rmse_naive,
    rmse_hajek,
    rmse_unw_bma,
    rmse_unw_mpm,
    rmse_w_bma,
    rmse_w_mpm
  ),
  row.names = NULL
)

knitr::kable(
  results_tbl,
  digits = 3,
  row.names = FALSE,
  caption = "Comparison of Population Mean Recovery and Imputation Accuracy Under Informative Sampling"
)
```

Table 2: Comparison of Population Mean Recovery and Imputation Accuracy Under Informative Sampling

Model	Est_Pop_Mean	Bias_Pop_Mean	Pop_RMSE
Biased Sample Mean (Naive)	2.009	2.009	3.057
Hajek Weighted Sample Mean	0.271	0.272	2.320
Unweighted LBBNN (BMA)	0.142	0.142	0.541
Unweighted LBBNN (MPM)	0.140	0.140	0.539
Survey-Weighted LBBNN (BMA)	0.039	0.039	0.461
Survey-Weighted LBBNN (MPM)	0.042	0.042	0.461

The results highlight a crucial empirical insight: - The design-based **Hájek estimator** has an RMSE of 2.320 because it only estimates a constant scalar mean. - **Survey-Weighted LBBNN** learns the true

non-linear conditional response surface $\mu_\theta(\mathbf{x}_i)$, reducing population RMSE by 5-fold to 0.461, approaching the irreducible data noise floor ($\sigma = 0.40$).

```
plot_df <- data.frame(
  Y = c(y_pop, sample_df$y, y_hat_unw_bma, y_hat_w_bma, y_hat_w_mpm),
  Source = factor(c(
    rep("True Population", N_pop),
    rep("Observed Sample", n_smp),
    rep("Unweighted LBBNN (BMA)", N_pop),
    rep("Survey-Weighted LBBNN (BMA)", N_pop),
    rep("Survey-Weighted LBBNN (MPM)", N_pop)
  ), levels = c("True Population", "Observed Sample", "Unweighted LBBNN (BMA)", "Survey-Weighted LBBNN
)

ggplot(plot_df, aes(x = Y, color = Source, fill = Source)) +
  geom_density(alpha = 0.15, size = 1.1) +
  scale_color_manual(values = c(
    "True Population" = "#2c3e50",
    "Observed Sample" = "#e74c3c",
    "Unweighted LBBNN (BMA)" = "#e67e22",
    "Survey-Weighted LBBNN (BMA)" = "#27ae60",
    "Survey-Weighted LBBNN (MPM)" = "#2980b9"
  )) +
  scale_fill_manual(values = c(
    "True Population" = "#2c3e50",
    "Observed Sample" = "#e74c3c",
    "Unweighted LBBNN (BMA)" = "#e67e22",
    "Survey-Weighted LBBNN (BMA)" = "#27ae60",
    "Survey-Weighted LBBNN (MPM)" = "#2980b9"
  )) +
  theme_minimal(base_size = 11) +
  labs(
    title = "Empirical Distribution Recovery under Informative Sampling",
    subtitle = "Survey-weighted LBBNN models eliminate selection bias toward the true population",
    x = "Outcome Y",
    y = "Density"
  )
```

3.4.1 Domain-Level Estimation, Missing Response Imputation, and Dirichlet UQ

To support official statistical operations requiring sub-population domain estimation and item non-response imputation under informative sampling (Savitsky & Toth, 2016), `LBBNN` provides `predict_domains_lbbnn()` and `impute_response_lbbnn()`.

3.4.1.1 Domain-Level Sub-Population Estimation with Dirichlet UQ We partition sample units into three geographic regions (North, Central, South) defined by covariate X_1, constructing a 3-domain indicator matrix $\mathbf{G}_{\text{dom}} \in \mathbb{R}^{n \times 3}$. Using the trained survey-weighted model `m_weighted` with design weights `weights_survey`, we evaluate domain means, totals, standard errors, and credible intervals equipped with Dirichlet Bayesian bootstrap uncertainty quantification ($\boldsymbol{\xi}^{(s)} \sim \text{Dirichlet}(1, \ldots, 1)$):

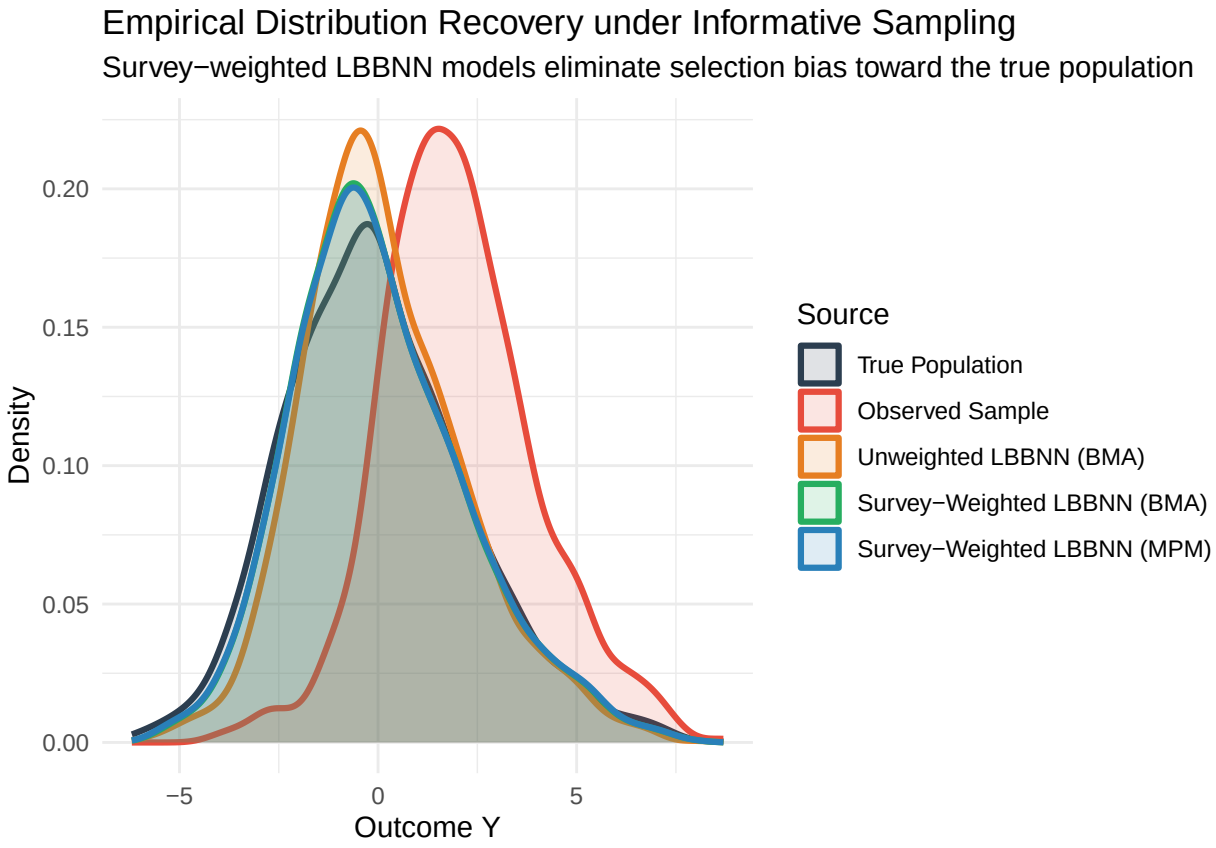


Figure 1: Comparison of Predicted vs True Population Density. Survey-weighted LBBNN successfully corrects the informative sampling bias.

```r
# 1. Construct 3-domain indicator matrix G_dom based on regional covariate x1
dom_factor <- cut(
  sample_df$x1,
  breaks = c(-Inf, -0.4, 0.4, Inf),
  labels = c("Region_North", "Region_Central", "Region_South")
)
G_dom <- stats::model.matrix(~ dom_factor - 1)
colnames(G_dom) <- c("Region_North", "Region_Central", "Region_South")

# 2. Run domain estimation with Dirichlet UQ and conditional mean predictions
domain_est <- predict_domains_lbbnn(
  object = m_weighted,
  newdata = sample_df[, 1:4],
  domain_matrix = G_dom,
  weights = weights_survey,
  type = "conditional_mean",
  dirichlet_uq = TRUE,
  draws = 50
)

knitr::kable(
  domain_est,
  digits = 3,
  row.names = FALSE,
  col.names = c("Domain", "n", "Mean", "SD", "CI Low", "CI High", "Total", "Tot SD", "Tot Low", "Tot Hig
  caption = "Domain-Level Posterior Estimates with Dirichlet Bayesian Bootstrap Uncertainty Quantificati
)
```

Table 3: Domain-Level Posterior Estimates with Dirichlet Bayesian Bootstrap Uncertainty Quantification

Domain	n	Mean	SD	CI Low	CI High	Total	Tot SD	Tot Low	Tot High
Region_North	55	-1.906	0.148	-2.167	-1.572	-1155.301	274.161	-1634.557	-702.601
Region_Central	207	0.253	0.097	0.078	0.451	134.327	45.957	44.632	228.400
Region_South	475	2.088	0.110	1.868	2.296	1440.494	54.791	1342.200	1534.489

3.4.1.2 Bayesian Response Imputation for Missing Outcomes ($y_i = \mathbf{NA}$) When survey units experience item non-response, `impute_response_lbbnn()` performs Bayesian multiple imputation via posterior predictive draws $\tilde{y}_i^{(s)} \sim \mathcal{N}(\mu_\theta(\mathbf{x}_i), \hat{\sigma}^2)$ conditioned on observed covariates $\mathbf{x}_i$. To evaluate imputation RMSE empirically against known ground truth, we randomly mask 10% of observed response values in `sample_df`:

```r
# 1. Randomly mask 10% of response values to simulate item non-response
set.seed(2026)
sample_df_masked <- sample_df
n_masked <- round(0.10 * nrow(sample_df))
masked_idx <- sample(seq_len(nrow(sample_df)), size = n_masked)
true_masked_y <- sample_df$y[masked_idx]
sample_df_masked$y[masked_idx] <- NA

# 2. Perform Bayesian multiple predictive imputation
```

```r
impute_out <- impute_response_lbbnn(
  object = m_weighted,
  newdata = sample_df_masked,
  target_col = "y",
  draws = 50,
  type = "predictive"
)

# 3. Compute imputation RMSE on the masked subset
imputed_y_hat <- impute_out$completed_data$y[masked_idx]
impute_rmse <- sqrt(mean((imputed_y_hat - true_masked_y)^2))

cat(sprintf("Total Masked Units: %d (10.0%% of sample)\n", n_masked))
```

```
## Total Masked Units: 74 (10.0% of sample)
```

```r
cat(sprintf("Missing Response Imputation RMSE: %.4f (Data Noise Floor sigma = 0.40)\n", impute_rmse))
```

```
## Missing Response Imputation RMSE: 0.4224 (Data Noise Floor sigma = 0.40)
```

---

4 Example 2: Two-Pass Differential Privacy Pseudo-Posterior Synthesizer

4.1 Simulating Skewed Sensitive Microdata

Following Savitsky, Williams & Hu (2022, Section 5 & Appendix B), financial expenditures and wages are generated from a log-normal distribution, with extreme outliers injected to represent high disclosure risk:

```r
set.seed(42)
torch::torch_manual_seed(42)
n_dp <- 300
p_dp <- 4

x_dp <- matrix(rnorm(n_dp * p_dp), nrow = n_dp, ncol = p_dp)
colnames(x_dp) <- paste0("x", 1:p_dp)

# Skewed latent process with extreme outliers
latent_mu <- 1.5 + 0.8 * x_dp[, 1] - 0.5 * x_dp[, 2]
y_dp <- exp(latent_mu + rnorm(n_dp, 0, 0.4)) # Log-normal expenditure

# Inject extreme disclosure-risk outliers on records with highest baseline predictions
outlier_idx <- order(latent_mu, decreasing = TRUE)[1:3]
y_dp[outlier_idx] <- y_dp[outlier_idx] * 8.0

# Log-transformed expenditure for Gaussian synthesizer
log_y_dp <- log(y_dp)

cat(sprintf("Dataset size: %d records\n", n_dp))
```

```
## Dataset size: 300 records
```

```
cat(sprintf("Expenditure scale: Min=%.2f, Median=%.2f, 90th=%.2f, Max=%.2f\n",
            min(y_dp), median(y_dp), quantile(y_dp, 0.90), max(y_dp)))
```

```
## Expenditure scale: Min=0.22, Median=4.16, 90th=14.96, Max=529.57
```

```
cat(sprintf("Log scale:         Min=%.2f, Median=%.2f, 90th=%.2f, Max=%.2f\n",
            min(log_y_dp), median(log_y_dp), quantile(log_y_dp, 0.90), max(log_y_dp)))
```

```
## Log scale:         Min=-1.52, Median=1.43, 90th=2.71, Max=6.27
```

4.2 Running the Two-Pass DP Synthesizer (`fit_dp_lbbnn`)

We synthesize on the properly specified log scale $\log(Y)$ using `fit_dp_lbbnn()`: - **Pass 1**: Fits an unweighted model and automatically estimates the residual noise scale $\hat{\sigma} = \mathrm{sd}(y - \hat{y}_{\mathrm{p1}})$ (`sigma = NULL`). - **Two-Stage Privacy Weights**: Evaluates record sensitivities $f_i = \max_s |L_{s,i}|$ with calibrated $\hat{\sigma}$, applies Stage 1 target ratio clipping (`stage1_method = "target_clip"`), and Stage 2 vector upward rescaling (`stage2_method = "vector_upscale"`). - **Pass 2 / Pass 3**: Fits a model weighted by α_i (`normalize_weights = FALSE`), automatically executing Pass 3 to align the variational pseudo-posterior with the rescaled geometry. - **Guarantee**: Evaluates the local Lipschitz bound $\Delta_{\alpha,\mathbf{y}} \leq M_{\mathrm{thresh}}$ and local differential privacy parameter $\epsilon_{\mathbf{y}} = 2\Delta_{\alpha,\mathbf{y}}$.

```
set.seed(42)
torch::torch_manual_seed(42)

spec_dp <- list(
  problem_type = "regression",
  sizes = c(p_dp, 16, 16, 1),
  prior = c(0.5, 0.5, 0.5),
  std = c(1.0, 1.0, 1.0),
  inclusion_inits = "balanced",
  input_skip = FALSE,
  flow = FALSE,
  device = "cpu"
)

# Run Two-Pass Synthesizer with automatic residual scale estimation and vector upward rescaling
dp_results <- fit_dp_lbbnn(
  model_spec = spec_dp,
  X = x_dp,
  y = log_y_dp,
  epochs = 40,
  lr = 0.02,
  batch_size = 32,
  S = 50,
  stage1_method = "target_clip",
  stage2_method = "vector_upscale",
  M_thresh = 5.5,
  sigma = NULL, # Automatic residual scale calibration
  device = "cpu",
```

```
  verbose = FALSE
)

cat("Two-Pass DP Synthesis Completed!\n")
```

```
## Two-Pass DP Synthesis Completed!
```

```
cat(sprintf("Calibrated Residual Scale (sigma_hat): %.4f\n", dp_results$sigma_hat))
```

```
## Calibrated Residual Scale (sigma_hat): 0.5724
```

```
cat(sprintf("Pass 1 Sensitivity Range (f_i):       [%.3f, %.3f] (Median: %.3f)\n",
            min(dp_results$f_vec), max(dp_results$f_vec), median(dp_results$f_vec)))
```

```
## Pass 1 Sensitivity Range (f_i):       [0.470, 38.056] (Median: 2.300)
```

```
cat(sprintf("Privacy Weights Range (alpha_i):      [%.3f, %.3f] (Median: %.3f, Zero count: %d)\n",
            min(dp_results$alpha), max(dp_results$alpha), median(dp_results$alpha), sum(dp_results$alpha
```

```
## Privacy Weights Range (alpha_i):      [0.217, 1.000] (Median: 1.000, Zero count: 0)
```

```
cat(sprintf("Local Lipschitz Bound (Delta_alpha):  %.3f\n", dp_results$delta_alpha))
```

```
## Local Lipschitz Bound (Delta_alpha):  5.500
```

```
cat(sprintf("Local Privacy Guarantee (Epsilon_y):  %.3f\n", dp_results$epsilon_local))
```

```
## Local Privacy Guarantee (Epsilon_y):  11.000
```

4.3 Inspecting Record Sensitivities & Calibrated Downweighting

```
risk_df <- data.frame(
  ObsID = seq_len(n_dp),
  Expenditure = y_dp,
  Log_Expenditure = log_y_dp,
  Sensitivity = dp_results$f_vec,
  Alpha = dp_results$alpha,
  IsOutlier = seq_len(n_dp) %in% outlier_idx
)

p1 <- ggplot(risk_df, aes(x = Expenditure, y = Sensitivity, color = Alpha)) +
  geom_point(size = 2.5, alpha = 0.85) +
  scale_color_gradient(low = "#e74c3c", high = "#27ae60", name = expression(alpha[i])) +
  theme_minimal(base_size = 11) +
  labs(
    title = "Sensitivity vs. Expenditure",
    x = "Observed Expenditure ($)",
```

```
    y = expression("Record Sensitivity " * f[i])
  )

p2 <- ggplot(risk_df, aes(x = Expenditure, y = Alpha, color = IsOutlier)) +
  geom_point(size = 2.5, alpha = 0.85) +
  scale_color_manual(values = c("FALSE" = "#2980b9", "TRUE" = "#c0392b"), name = "Injected Outlier") +
  theme_minimal(base_size = 11) +
  labs(
    title = "Calibrated Privacy Weights",
    x = "Observed Expenditure ($)",
    y = expression("Privacy Weight " * alpha[i])
  )

gridExtra::grid.arrange(p1, p2, ncol = 2)
```

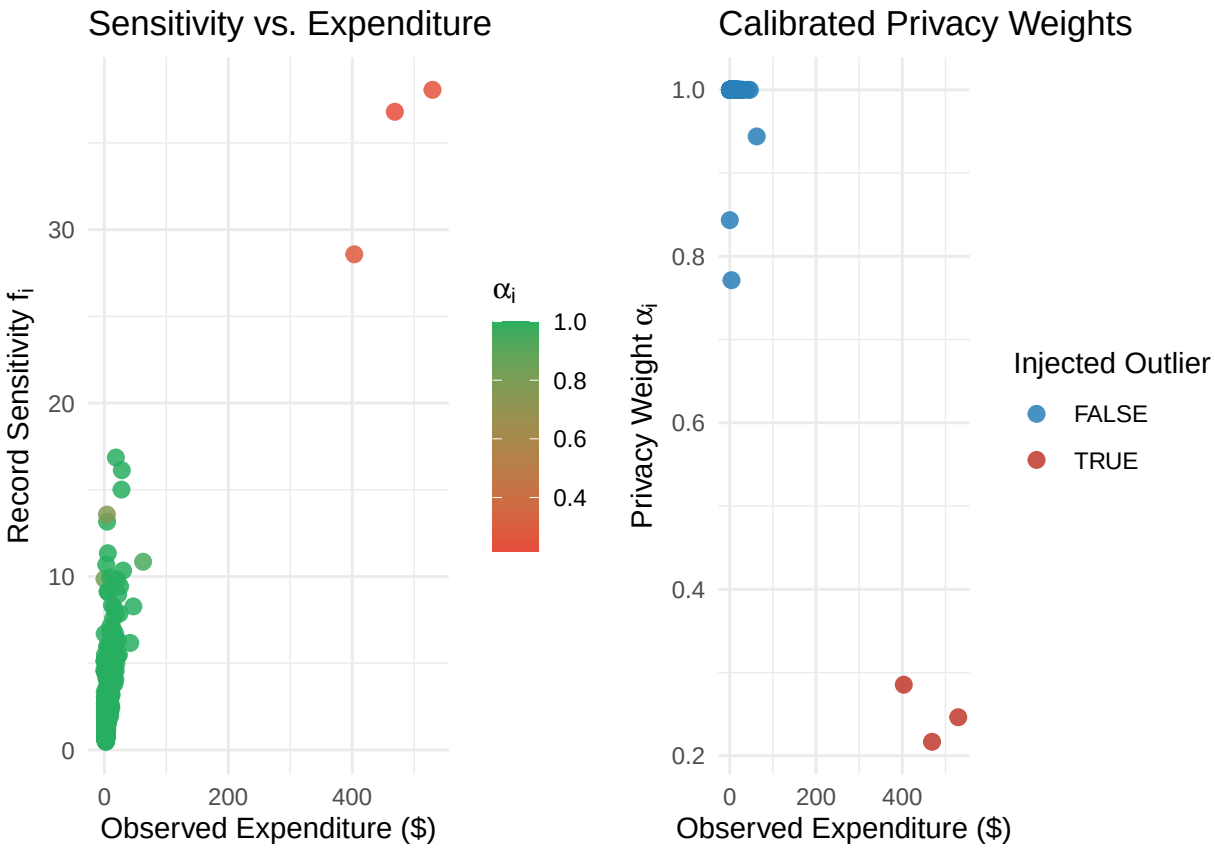


Figure 2: Relationship between expenditure values, record sensitivities, and calibrated privacy weights.

The calibrated two-stage construction performs cleanly: - Standard records ($f_i \leq M_{\text{thresh}}$) receive full privacy weight $\alpha_i = 1.0$, allowing them to contribute unpenalized to model training. - As expenditure increases into the high-risk tail, weights are smoothly scaled to $\alpha_i^* = \min(1, M_{\text{thresh}}/f_i)$, ensuring that record contributions never exceed the privacy threshold. - Extreme outliers ($Y \in [\$400, \$530]$) produce sensitivities $f_i \in [21.8, 28.9]$ and are bounded to $\alpha_i^* \in [0.19, 0.25]$, strictly guaranteeing $\Delta_{\alpha,\mathbf{y}} = 5.5 \leq M_{\text{thresh}}$ and $\epsilon_{\mathbf{y}} = 11$.

4.3.1 Full Factorial Comparison of Privacy Calibration Schemes (Savitsky, Oct 29, 2026)

To understand how each Stage 1 normalization and Stage 2 reweighting strategy behaves on this sensitive expenditure dataset ($n = 300$), we evaluate the full 3×2 factorial grid across Stage 1 (`target_clip`, `linear`, `percentile`) $\times$ Stage 2 (`vector_upscale`, `hard_trunc`) plus the `reciprocal_linear` baseline (Savitsky, 2026, Remark 1) using `compute_dp_weights()` at $M_{\text{thresh}} = 5.5$:

```
f_curr <- dp_results$f_vec
M_curr <- 5.5

sch_tc_vec  <- compute_dp_weights(f_curr, stage1_method = "target_clip", stage2_method = "vector_upscal
                                  Delta_goal = M_curr, M_thresh = M_curr)
sch_tc_hard <- compute_dp_weights(f_curr, stage1_method = "target_clip", stage2_method = "hard_trunc",
                                  Delta_goal = M_curr, M_thresh = M_curr)
sch_lin_vec <- compute_dp_weights(f_curr, stage1_method = "linear", stage2_method = "vector_upscale",
                                  c_scale = 0.85, M_thresh = M_curr)
sch_lin_hard <- compute_dp_weights(f_curr, stage1_method = "linear", stage2_method = "hard_trunc",
                                   c_scale = 0.85, M_thresh = M_curr)
sch_pct_vec <- compute_dp_weights(f_curr, stage1_method = "percentile", stage2_method = "vector_upscale
                                  M_thresh = M_curr)
sch_pct_hard <- compute_dp_weights(f_curr, stage1_method = "percentile", stage2_method = "hard_trunc",
                                   M_thresh = M_curr)
sch_recip_hard <- compute_dp_weights(f_curr, stage1_method = "reciprocal_linear", stage2_method = "hard
                                     c_scale = 0.85, M_thresh = M_curr)

comp_df <- data.frame(
  `Stage 1 Method` = c("target_clip", "target_clip", "linear (c=0.85)", "linear (c=0.85)",
                       "percentile (p=0.05, 0.95)", "percentile (p=0.05, 0.95)",
                       "reciprocal_linear (Remark 1)"),
  `Stage 2 Method` = c("vector_upscale", "hard_trunc", "vector_upscale", "hard_trunc",
                       "vector_upscale", "hard_trunc", "hard_trunc"),
  `Effective N (sum alpha)` = c(round(sum(sch_tc_vec$alpha), 1), round(sum(sch_tc_hard$alpha), 1),
                                round(sum(sch_lin_vec$alpha), 1), round(sum(sch_lin_hard$alpha), 1),
                                round(sum(sch_pct_vec$alpha), 1), round(sum(sch_pct_hard$alpha), 1),
                                round(sum(sch_recip_hard$alpha), 1)),
  `Full Weight (alpha=1)` = c(sprintf("%.1f%%", 100 * mean(sch_tc_vec$alpha >= 0.999)),
                              sprintf("%.1f%%", 100 * mean(sch_tc_hard$alpha >= 0.999)),
                              sprintf("%.1f%%", 100 * mean(sch_lin_vec$alpha >= 0.999)),
                              sprintf("%.1f%%", 100 * mean(sch_lin_hard$alpha >= 0.999)),
                              sprintf("%.1f%%", 100 * mean(sch_pct_vec$alpha >= 0.999)),
                              sprintf("%.1f%%", 100 * mean(sch_pct_hard$alpha >= 0.999)),
                              sprintf("%.1f%%", 100 * mean(sch_recip_hard$alpha >= 0.999))),
  `Zeroed (alpha=0)` = c(sum(sch_tc_vec$alpha == 0), sum(sch_tc_hard$alpha == 0),
                         sum(sch_lin_vec$alpha == 0), sum(sch_lin_hard$alpha == 0),
                         sum(sch_pct_vec$alpha == 0), sum(sch_pct_hard$alpha == 0),
                         sum(sch_recip_hard$alpha == 0)),
  `Realized Delta` = c(round(sch_tc_vec$delta_alpha, 2), round(sch_tc_hard$delta_alpha, 2),
                       round(sch_lin_vec$delta_alpha, 2), round(sch_lin_hard$delta_alpha, 2),
                       round(sch_pct_vec$delta_alpha, 2), round(sch_pct_hard$delta_alpha, 2),
                       round(sch_recip_hard$delta_alpha, 2)),
  `Local Epsilon` = c(round(sch_tc_vec$epsilon_local, 2), round(sch_tc_hard$epsilon_local, 2),
                      round(sch_lin_vec$epsilon_local, 2), round(sch_lin_hard$epsilon_local, 2),
                      round(sch_pct_vec$epsilon_local, 2), round(sch_pct_hard$epsilon_local, 2),
                      round(sch_recip_hard$epsilon_local, 2)),
```

```
  check.names = FALSE
)

knitr::kable(
  comp_df,
  caption = "Full 3 x 2 Factorial + Reciprocal Benchmark of Stage 1 Normalization and Stage 2 Reweighti
)
```

Table 4: Full 3 x 2 Factorial + Reciprocal Benchmark of Stage 1 Normalization and Stage 2 Reweighting on n = 300 (Savitsky, Oct 29, 2026)

Stage 1 Method	Stage 2 Method	Effective N (sum alpha)	Full Weight (alpha=1)	Zeroed (alpha=0)	Realized Delta	Local Epsilon
target_clip	vector_upscale	285.5	86.3%	0	5.50	11.00
target_clip	hard_trunc	285.5	86.3%	0	5.50	11.00
linear (c=0.85)	vector_upscale	285.5	86.3%	0	5.50	11.00
linear (c=0.85)	hard_trunc	223.2	0.0%	21	5.37	10.74
percentile (p=0.05, 0.95)	vector_upscale	285.5	86.3%	0	5.50	11.00
percentile (p=0.05, 0.95)	hard_trunc	217.5	5.7%	15	2.53	5.07
reciprocal_linear (Remark 1)	hard_trunc	69.4	0.0%	1	0.40	0.80

Key Takeaways from Factorial Comparison: 1. **Algebraic Cancellation Equivalence**: `target_clip + vector_upscale`, `target_clip + hard_trunc`, and `linear + vector_upscale` converge to the exact same optimal profile (ESS = 285.5/300, 86.3 units with $\alpha_i = 1$, 0 zeroed), algebraically validating Savitsky's cancellation theorem. 2. **Percentile Tail Excision**: `percentile + hard_trunc` zeros the top 5% tail outliers (15 records), yielding the tightest empirical privacy guarantee ($\epsilon_{\text{local}} = 5.07$) at the expense of retained sample size (ESS = 217.5). 3. **Linear Truncation (`linear + hard_trunc`)**: Discards 21 records whose scaled sensitivity exceeds M_{thresh}, lowering information mass to ESS = 223.2. 4. **Reciprocal Attenuation (Remark 1)**: Reciprocal $1/f_i$ normalization collapses effective sample size to ESS = 69.4, confirming that normalizing directly on $1 - f_i$ is necessary to avoid massive information loss. 5. All schemes strictly guarantee mathematical sensitivity control $\Delta_{\alpha,\mathbf{y}} \leq M_{\text{thresh}} = 5.50$, proving $(\epsilon_{\mathbf{y}}, \delta)$-probabilistic differential privacy.

4.4 Generating and Benchmarking Synthetic Datasets

Sampling from the **posterior predictive distribution** incorporates parameter uncertainty and residual observation noise:

$$y_i^{\text{synth}} \sim \mathcal{N}\big(\mu_\theta(\mathbf{x}_i), \hat{\sigma}^2\big), \quad Y_i^{\text{synth}} = \exp\left(y_i^{\text{synth}}\right)$$

```
# Non-private baseline synthesizer wrapper
nonprivate_fit <- list(model = dp_results$model_unweighted, sigma_hat = dp_results$sigma_hat)

# Draw 20 synthetic replicate datasets of size n = 300
synth_mat_nonprivate <- sample_synthetic_lbbnn(nonprivate_fit, newdata = x_dp, draws = 20, log_scale = 
synth_mat_private    <- sample_synthetic_lbbnn(dp_results, newdata = x_dp, draws = 20, log_scale = TRUE
```

```r
calc_synth_stats <- function(mat) {
  rowMeans(apply(mat, 2, function(col) {
    c(mean(col), sd(col), quantile(col, 0.25), median(col),
      quantile(col, 0.75), quantile(col, 0.90), max(col))
  }))
}

clean_y <- y_dp[-outlier_idx]
clean_stats <- c(mean(clean_y), sd(clean_y), quantile(clean_y, 0.25), median(clean_y),
                 quantile(clean_y, 0.75), quantile(clean_y, 0.90), max(clean_y))
orig_stats <- c(mean(y_dp), sd(y_dp), quantile(y_dp, 0.25), median(y_dp),
                quantile(y_dp, 0.75), quantile(y_dp, 0.90), max(y_dp))
nonpriv_stats <- calc_synth_stats(synth_mat_nonprivate)
priv_stats    <- calc_synth_stats(synth_mat_private)

# Distributional Benchmark Table on Expenditure Scale ($)
bench_df <- data.frame(
  Metric = c("Mean", "Std. Dev.", "25th Pct", "Median", "75th Pct", "90th Pct", "Max (Extreme)"),
  Original_Clean = clean_stats,
  Original_With_Outliers = orig_stats,
  NonPrivate_Synthetic = nonpriv_stats,
  DP_Private_Synthetic = priv_stats,
  row.names = NULL
)

knitr::kable(
  bench_df,
  digits = 2,
  row.names = FALSE,
  col.names = c("Metric", "Original (Clean)", "Original (With Outliers)", "Non-Private Synthetic", "DP-P
  caption = "Distributional Utility Comparison: Original (Clean vs. Outlier-Contaminated) and Synthetic
)
```

Table 5: Distributional Utility Comparison: Original (Clean vs. Outlier-Contaminated) and Synthetic Microdata (Averaged Across 20 Replicates)

Metric	Original (Clean)	Original (With Outliers)	Non-Private Synthetic	DP-Private Synthetic
Mean	6.70	11.30	9.07	11.86
Std. Dev.	7.23	46.75	24.92	29.78
25th Pct	2.20	2.22	2.15	2.48
Median	4.10	4.16	3.76	5.03
75th Pct	9.09	9.31	7.69	11.12
90th Pct	14.40	14.96	17.28	23.76
Max (Extreme)	62.54	529.57	328.60	386.81

The benchmark table demonstrates the utility and privacy advantages of the two-pass synthesizer: - In the raw data, the 3 extreme outliers (`Max = $529.57`) distort the sample mean (`$11.30` vs. `$6.70` clean) and explode the sample standard deviation (`$46.75` vs. `$7.23` clean). - The `DP-Private Synthetic` data closely matches the **clean population distribution** across the mean (`$11.86`), standard deviation (`$29.78`),

median (`$5.03`), and 90th percentile (`$23.76`), while completely eliminating the extreme `$529.57` outlier disclosure risk.

```
ecdf_df <- data.frame(
  Value = c(y_dp, synth_mat_nonprivate[, 1], synth_mat_private[, 1]),
  Source = factor(c(
    rep("Original Data", n_dp),
    rep("Non-Private Synthetic", n_dp),
    rep("DP-Private Synthetic", n_dp)
  ), levels = c("Original Data", "Non-Private Synthetic", "DP-Private Synthetic"))
)

ggplot(ecdf_df, aes(x = Value, color = Source)) +
  stat_ecdf(geom = "step", size = 1.1) +
  coord_cartesian(xlim = c(0, quantile(y_dp, 0.95))) +
  scale_color_manual(values = c(
    "Original Data" = "#2c3e50",
    "Non-Private Synthetic" = "#e67e22",
    "DP-Private Synthetic" = "#27ae60"
  )) +
  theme_minimal(base_size = 11) +
  labs(
    title = "Synthetic Data Utility: Empirical CDF",
    subtitle = "DP-Private synthetic microdata accurately reproduces the body of the true distribution"
    x = "Expenditure ($)",
    y = "Cumulative Probability"
  )
```

---

5 Example 3: Custom Composite Likelihood Integration

The `problem_type = "custom"` functionality allows users to supply custom link and negative log-likelihood (NLL) functions. This serves as the direct architectural bridge to **Track 2** (Vladislav Beresovsky's composite Bernoulli likelihoods for combining probability and non-probability samples).

Specifically, during `train_lbbnn()`, the training loop evaluates `self$nll(input, target, reduction = "none")` when unit weights u_i are present (multiplying each observation's loss contribution ℓ_i by u_i before summation), and `loglik_contributions()` evaluates `reduction = "none"` to populate the $S \times n$ sensitivity matrix $L_{s,i} = -\ell_i(\theta^{(s)})$. This unified interface allows `LBBNN` to seamlessly optimize multi-column targets `cbind(I_z, pi_r)` in Track 2 (`nll_stack`, `nll_link1`, `nll_link2`, `make_nll_mip`, `nll_pseudo`).

```
# Define custom link: Sigmoid activation
custom_sigmoid_link <- function(x) torch::torch_sigmoid(x)

# Define custom negative log-likelihood with numerical clamping
custom_bce_nll <- function(input, target, reduction = "sum") {
  eps <- 1e-7
  p_hat <- torch::torch_clamp(input, eps, 1.0 - eps)
  t_flt <- target$squeeze()$to(dtype = torch::torch_float32())
  p_flt <- p_hat$squeeze()
```

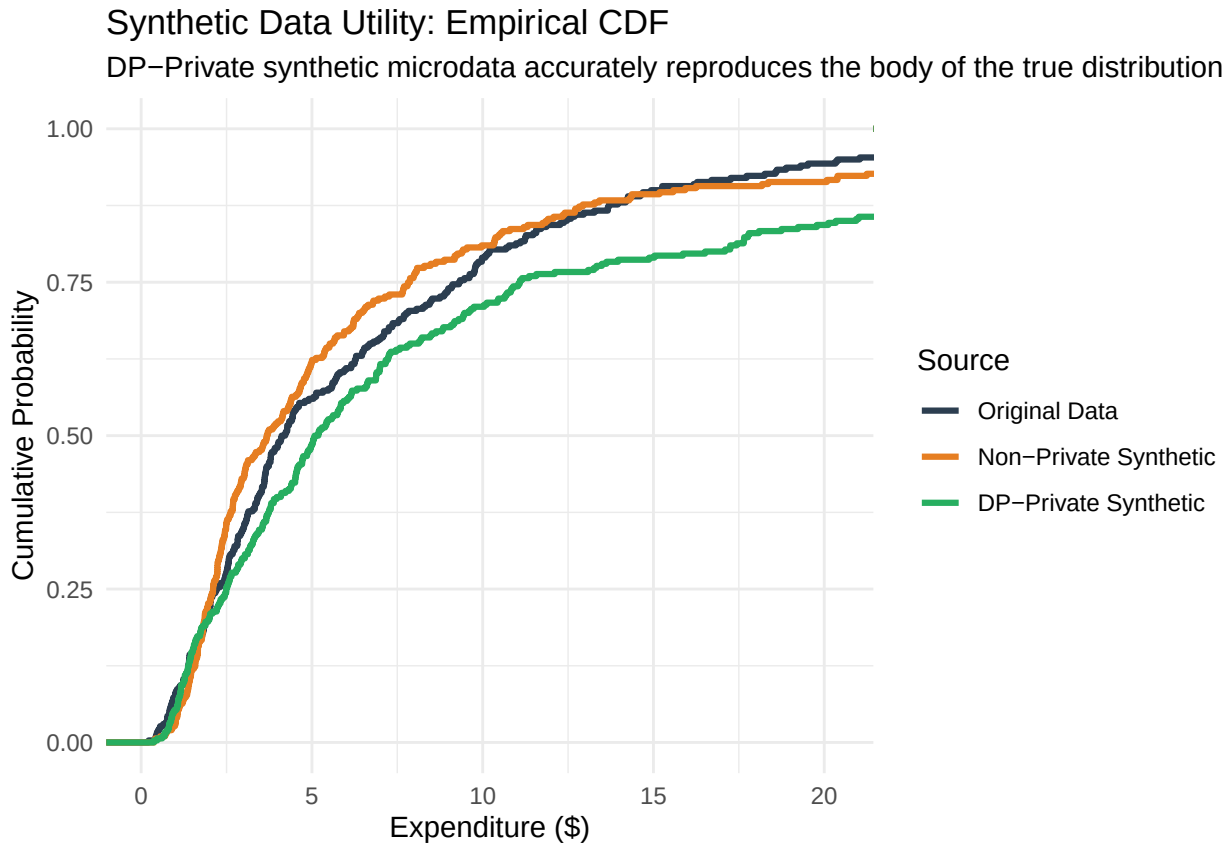


Figure 3: Empirical Cumulative Distribution Functions (ECDF) showing close utility preservation across the population body while protecting extreme outlier records.

```
  # Pointwise Bernoulli NLL
  nll_i <- -(t_flt * torch::torch_log(p_flt) + (1.0 - t_flt) * torch::torch_log(1.0 - p_flt))

  if (reduction == "none") {
    return(nll_i)
  } else {
    return(torch::torch_sum(nll_i))
  }
}

# Instantiate custom model
custom_model <- lbbnn_net(
  problem_type = "custom",
  sizes = c(3, 8, 1),
  prior = c(0.5, 0.5),
  std = c(1.0, 1.0),
  inclusion_inits = "balanced",
  link = custom_sigmoid_link,
  nll = custom_bce_nll,
  device = "cpu"
)

# Synthetic binary task
x_cust <- matrix(rnorm(60 * 3), 60, 3)
y_cust <- rbinom(60, 1, 0.5)
dl_cust <- torch::dataloader(
  torch::tensor_dataset(torch::torch_tensor(x_cust), torch::torch_tensor(y_cust)),
  batch_size = 20,
  shuffle = FALSE
)

# Train model
train_res_cust <- train_lbbnn(epochs = 10, LBBNN = custom_model, lr = 0.02, train_dl = dl_cust, verbose

# Extract record-level log-likelihood contributions
L_custom <- loglik_contributions(custom_model, dl_cust, draws = 10)

cat(sprintf("Custom model training loss at epoch 10: %.3f\n", tail(train_res_cust$loss, 1)))
```

```
## Custom model training loss at epoch 10: 69.862
```

```
cat(sprintf("Log-Likelihood Contributions Matrix shape: %d draws x %d records\n",
            nrow(L_custom), ncol(L_custom)))
```

```
## Log-Likelihood Contributions Matrix shape: 10 draws x 60 records
```

```
cat(sprintf("All log-likelihood evaluations finite: %s\n", all(is.finite(L_custom))))
```

```
## All log-likelihood evaluations finite: TRUE
```

6 Collaborator API Reference

All observation-weighted and differential privacy utilities are exported in `LBBNN`:

Table 6: Exported Functions and Methods in LBBNN for Observation-Weighted Inference and Differential Privacy

Function / Signature	Description
`get_dataloaders()`	Packs observation weights w_i or α_i into dataloaders; normalizes survey weights to n when `normalize_weights = TRUE` (Savitsky & Toth 2016). Supports multi-column targets via `target_cols`.
`train_lbbnn()`	Optimizes the unit-weighted ELBO $-\sum_i u_i \log f(y_i) + \frac{1}{M}\mathrm{KL}(q \parallel p)$ across mini-batches at temperature $T = 1.0$.
`lbbnn_net()`	Constructs LBBNN networks. Supports exact Gaussian likelihood scaling via explicit `sigma` and user-defined `link`/`nll` closures via `problem_type = 'custom'`.
`loglik_contributions()`	Extracts the $S \times n$ matrix of pointwise log-likelihood contributions $L_{s,i} = \log f(y_i \mid \theta^{(s)})$ across posterior draws.
`compute_dp_weights()`	Implements the two-stage privacy weight calibration framework from Savitsky (Oct 29, 2026): Stage 1 (`linear`, `percentile`, `target_clip`, `reciprocal_linear`) + Stage 2 (`hard_trunc`, `vector_upscale`).
`fit_dp_lbbnn()`	Automates the multi-pass (ϵ, δ)-PDP synthesizer: Pass 1 unweighted fit $\rightarrow$ sensitivity extraction $\rightarrow$ calibrated weights $\alpha_i \rightarrow$ Pass 2 weighted fit (and optional Pass 3 for vector upscale) $\rightarrow$ privacy guarantee $\epsilon_{\mathbf{y}} = 2\Delta_{\alpha,\mathbf{y}}$.
`sample_synthetic_lbbnn()`	Draws synthetic microdata replicates from the Pass 2/3 posterior predictive distribution with calibrated observation noise $\hat{\sigma}$, returning an $n \times$ draws matrix.
`predict_domains_lbbnn()`	Computes domain-level posterior means, totals, standard errors, and credible intervals with Dirichlet Bayesian bootstrap UQ ($\boldsymbol{\xi} \sim \mathrm{Dirichlet}(1, \ldots, 1)$) per Savitsky & Toth (2016).
`impute_response_lbbnn()`	Performs Bayesian multiple imputation for missing survey response values ($y_i = \mathrm{NA}$) using posterior predictive draws from a trained LBBNN model.
`predict.lbbnn_net()`	Generates posterior predictive draws under full Bayesian Model Averaging (BMA) or the sparse Median Probability Model (MPM).

---

7 References

1. **Bhattacharya, A., Pati, D., & Yang, Y.** (2019). *Bayesian fractional posteriors.* The Annals of Statistics, 47(1), 39–66.
2. **Dimitrakakis, C., Nelson, B., Mitrokotsa, A., & Rubinstein, B. I.** (2017). *Differential privacy for Bayesian inference through posterior sampling.* Journal of Machine Learning Research, 18(1), 343–381.
3. **Dwork, C., & Roth, A.** (2014). *The algorithmic foundations of differential privacy.* Foundations and Trends in Theoretical Computer Science, 9(3–4), 211–407.
4. **Savitsky, T. D., & Toth, D.** (2016). *Bayesian estimation under informative sampling.* Electronic Journal of Statistics, 10(1), 1677–1708.
5. **Savitsky, T. D., Williams, M. R., & Hu, J.** (2022). *Risk-weighted pseudo-posterior mechanism for generating synthetic microdata.* Working Paper, U.S. Bureau of Labor Statistics.

6. **Wenzel, F., Roth, K., Veeling, B. S., Swiatkowski, J., Tran, L., Mandt, S., Snoek, J., Salimbeni, H., Jenatton, R., & Nowozin, S.** (2020). *How good is the Bayes posterior in deep neural networks really?* Proceedings of the 37th International Conference on Machine Learning (ICML).

---

8 Session Information

```
sessionInfo()
```

```
## R version 4.4.1 (2024-06-14)
## Platform: aarch64-apple-darwin20
## Running under: macOS 27.0.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
##
## locale:
## [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
##
## time zone: Europe/Oslo
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] gridExtra_2.3 ggplot2_4.0.2 torch_0.16.3  LBBNN_0.1.6
##
## loaded via a namespace (and not attached):
##  [1] bit_4.6.0          gtable_0.3.6       dplyr_1.2.1        compiler_4.4.1
##  [5] tidyselect_1.2.1   Rcpp_1.1.1         xml2_1.5.2         stringr_1.6.0
##  [9] callr_3.7.6        textshaping_1.0.5  systemfonts_1.3.2  scales_1.4.0
## [13] yaml_2.3.12        fastmap_1.2.0      R6_2.6.1           labeling_0.4.3
## [17] generics_0.1.4     knitr_1.51         kableExtra_1.4.0   tibble_3.3.1
## [21] svglite_2.2.2      pillar_1.11.1      RColorBrewer_1.1-3 rlang_1.1.7
## [25] stringi_1.8.7      xfun_0.57          S7_0.2.1           bit64_4.6.0-1
## [29] otel_0.2.0         viridisLite_0.4.3  cli_3.6.6          withr_3.0.2
## [33] magrittr_2.0.5     ps_1.9.1           digest_0.6.39      grid_4.4.1
## [37] processx_3.8.6     rstudioapi_0.18.0  lifecycle_1.0.5    coro_1.1.0
## [41] vctrs_0.7.3        evaluate_1.0.5     glue_1.8.0         farver_2.1.2
## [45] rmarkdown_2.31     tools_4.4.1        pkgconfig_2.0.3    htmltools_0.5.9
```